

# Laboratorio di Calcolo

## A.A. 2026/2027, Canale A-Dec

**Giacomo Artoni**, Lorenzo Caprini

Output formattato

# La funzione printf

`printf` (*print formatted*) scrive sul terminale. È dichiarata in `<stdio.h>`

```
printf(stringa_di_formato, arg1, arg2, ...);
```

- La **stringa di formato** contiene il testo da stampare.
- Al suo interno, ogni **specificatore** ( `%d` , `%f` , …) è un **segnaposto**.
- Gli argomenti successivi riempiono i segnaposto **nell'ordine**.
- Il numero e il **tipo** degli argomenti devono corrispondere ai segnaposto.

```
int    n = 3;
double m = 1.67e-27;

printf("Ho %d protoni, ognuno di massa %e kg\n",
       n, m);
```

```
Ho 3 protoni, ognuno di massa 1.670000e-27 kg
```

# Sequenze di *escape*

Alcuni caratteri non si possono scrivere direttamente in una stringa: si usa il **backslash** `\`.

| Sequenza | Significato |
|---|---|
| `\n` | A capo (*newline*) |
| `\t` | Tabulazione |
| `\\` | Il carattere `\` |
| `\"` | Le virgolette `"` |
| `\'` | L'apice `'` |
| `%%` | Il carattere `%` (in `printf`) |

```c
printf("Lui disse: \"ciao\"\n");
printf("C:\\percorso\\file\n");
printf("Sconto del 20%%\n");
printf("t\tx\n0.0\t1.0\n");
```

```
Lui disse: "ciao"
C:\percorso\file
Sconto del 20%
t       x
0.0     1.0
```

`%%` non è una sequenza di escape del C: è `printf` a interpretare `%` come inizio di un segnaposto.

# Gli specificatori di formato

Ogni tipo ha il suo specificatore: devono **corrispondere**.

| Specificatore | Tipo | Esempio di output |
|---|---|---|
| `%d` | `int` | `42` |
| `%u` | `unsigned int` | `4294967295` |
| `%ld` / `%lld` | `long` / `long long` | `5000050000` |
| `%f` | `double` (e `float`) | `3.141593` |
| `%e` | `double`, notazione scientifica | `2.998000e+08` |
| `%g` | `double`, la forma più compatta | `2.998e+08` |
| `%c` | `char` | `A` |
| `%s` | stringa | `Ciao` |
| `%x` / `%o` | `int` in esadecimale / ottale | `ff` / `377` |

- `%f` stampa per default **6 cifre decimali**.
- Un `float` passato a `printf` viene convertito in `double`: per entrambi si usa `%f`.
- `%x` e `%o` stampano le basi viste nella lezione 2: `printf("%x", 255)` dà `ff`.
- Attenzione: con `scanf` (prossimamente su questi schermi) le regole per `float` e `double` sono **diverse**!

# Specificatore sbagliato

```c
#include <stdio.h>

int main() {
    double x = 3.5;
    int n = 2;

    printf("x = %d\n", x);
    printf("n = %d, m = %d\n", n);
    return 0;
}
```

**Output imprevedibile:** a seconda della macchina si ottiene `0`, un numero enorme o un valore che cambia a ogni esecuzione. Non c'è modo di saperlo in anticipo.

```
$ gcc -Wall -Wextra -o sbagliato sbagliato.c
sbagliato.c:7:17: warning: format ‘%d’ expects
  argument of type ‘int’, but argument 2 has
  type ‘double’ [-Wformat=]
sbagliato.c:8:25: warning: format ‘%d’ expects
  a matching ‘int’ argument [-Wformat=]
```

- `printf` **non converte** il valore: legge i bit come se fossero del tipo indicato.
- Il programma compila, ma il risultato è **senza senso**.
- `-Wall` controlla i formati: **leggete i warning**!

# Ampiezza e precisione

Tra `%` e la lettera si possono indicare **ampiezza** minima del campo e **precisione**: `%[flag][ampiezza].[precisione]f`.

Supponiamo di avere `double x = 3.14159265;`

```
1   printf("%f\n",       x);
2   printf("%.2f\n",     x);
3   printf("%10.3f\n",   x);
4   printf("%-10.3f\n",  x);
5   printf("%010.3f\n",  x);
6   printf("%+.3f\n",    x);
```

```
1   3.141593
2   3.14
3         3.142
4   3.142
5   000003.142
6   +3.142
```

- `.3` : **cifre decimali** (arrotondate);

  `10` : **ampiezza** minima del campo.
- Flag: `-` allinea a **sinistra**, `0` riempie con **zeri**,

  `+` mostra il **segno**.
- Vale anche per gli interi: `%5d`, `%-5d`, `%05d`.

# Notazione scientifica

```c
double c = 2.998e8;     // [m/s]
double e = 1.602e-19;   // [C]

printf("%f\n", c);
printf("%e\n", c);
printf("%.3e\n", c);

printf("%f\n", e);
printf("%e\n", e);
printf("%g\n", e);
```

```
299800000.000000
2.998000e+08
2.998e+08
0.000000
1.602000e-19
1.602e-19
```

- `%e` stampa $2.998 \times 10^8$ come `2.998000e+08`.
- La carica dell'elettrone con `%f` diventa `0.000000`: le cifre significative sono **tutte dopo** la sesta cifra decimale!
- `%g` sceglie da solo tra `%f` ed `%e` e toglie gli zeri inutili.
- In fisica le grandezze spaziano su molti ordini di grandezza: usate `%e` o `%g`.

# Quante cifre stampare?

```c
printf("%.20f\n", 0.1);
printf("%.17g\n", 0.1);
printf("%.10f\n", 0.1f);
```

```
0.10000000000000000555
0.10000000000000001
0.1000000015
```

Chiedere più cifre di quelle significative mostra l'**errore di rappresentazione** (lezione 4): `0.1` non è esattamente rappresentabile in binario.

- Un `double` ha circa **16** cifre significative, un `float` circa **7**: le cifre oltre sono "rumore".
- La precisione con cui **stampate** un risultato deve riflettere la sua **incertezza**, come nelle relazioni di laboratorio.
- Una misura con incertezza di $0.01\ \mathrm{m}$ si stampa con `%.2f`, non con 6 decimali.

# Tabelle allineate

Con l'**ampiezza** si costruiscono tabelle leggibili.

```c
printf("%-8s %12s %10s\n",
       "Pianeta", "Massa [kg]", "R [km]");
printf("%-8s %12.3e %10.1f\n",
       "Terra", 5.972e24, 6371.0);
printf("%-8s %12.3e %10.1f\n",
       "Marte", 6.417e23, 3389.5);
printf("%-8s %12.3e %10.1f\n",
       "Giove", 1.898e27, 69911.0);
```

```
Pianeta    Massa [kg]      R [km]
Terra       5.972e+24      6371.0
Marte       6.417e+23      3389.5
Giove       1.898e+27     69911.0
```

- La stessa ampiezza nell'intestazione e nelle righe allinea le colonne.
- `%-8s` : testo allineato a **sinistra**, numeri a **destra**.
- Scegliete l'ampiezza in base al numero **più lungo**: `5.972e+24` occupa 9 caratteri.

# Output da leggere con Python

- Per fare un grafico (lezione 9) servono **colonne di numeri** separate da spazi.
- Le intestazioni vanno precedute da `#`: `np.loadtxt` le ignora.
- Stampate **abbastanza cifre** e niente testo sulle righe di dati.
- Con `>` l'output del programma finisce in un **file** invece che sul terminale.

```c
printf("# t[s]  x[m]\n");
printf("%.2f  %.4f\n", t1, x1);
printf("%.2f  %.4f\n", t2, x2);
printf("%.2f  %.4f\n", t3, x3);
```

```
$ ./moto > moto.dat
$ cat moto.dat
# t[s]  x[m]
0.50  2.7500
1.00  5.0000
1.50  7.7500
```

Con i **cicli** non avremo più bisogno di scrivere una `printf` per ogni riga!

# Errori tipici con printf

| Errore | Cosa succede |
|---|---|
| Specificatore sbagliato ( `%d` per un `double` ) | Valore senza senso; `-Wall` dà un warning `-Wformat` |
| Meno argomenti che segnaposto | Valori "spazzatura"; warning `expects a matching argument` |
| Più argomenti che segnaposto | Gli argomenti in più sono ignorati; warning `too many arguments for format` |
| `\n` dimenticato | Il prompt del terminale compare sulla stessa riga dell'output |
| `%` da solo nel testo | Interpretato come inizio di un segnaposto: si scrive `%%` |
| `%f` per numeri molto piccoli | Stampa `0.000000` : usate `%e` o `%g` |
| Virgola dimenticata tra gli argomenti | Errore di sintassi in compilazione |

Quasi tutti questi errori vengono segnalati da `gcc -Wall -Wextra` : un programma corretto compila **senza warning**.

# Esercizio

Cosa stampano queste istruzioni? Stiamo usando `[ ]` per poter vedere al meglio gli spazi:

```c
printf("[%5d]",   42);
printf("[%-5d]",  42);
printf("[%.2f]",  1.005);
printf("[%8.2e]", 12345.678);
printf("[%g]",    0.0001);
printf("[%x]",    31);
printf("[%5.1f%%]", 12.345);
```

| Istruzione | Output |
|---|---|
| `%5d` | `[   42]` |
| `%-5d` | `[42   ]` |
| `%.2f` | `[1.00]` (in memoria `1.005` vale `1.00499...` ) |
| `%8.2e` | `[1.23e+04]` |
| `%g` | `[0.0001]` |
| `%x` | `[1f]` |
| `%5.1f%%` | `[ 12.3%]` |